Comandos — ESP32-P4 v1.20.0

USB Host e registros ampliados — 1.20.0

Abra **USB / Registros** no menu web (`/usb`) para ver host, teclado, mouse, pendrive, listagem da raiz e fila de gravação no SD. As USB-A usam o controlador **high-speed dedicado / TinyUSB root port 1**, preservando GPIO27 do botão e a interface de programação. Não se inicia a pilha ESP-IDF USB Host em paralelo com TinyUSB.

Ligações e dispositivos suportados

- Jumper da base em **DEVICE**: ativa as **portas USB-A 2–4**, através do hub CH334. Esse nome é a marcação do seletor da placa; o firmware continua operando como host USB.
- Jumper em **HOST**: ativa a **porta USB-A 1 direta**. O hardware não oferece as quatro portas simultâneas nessa seleção. O firmware não movimenta o jumper.
- Teclado e mouse HID: drivers recebem relatórios, mostram conexão/remoção e contadores de atividade. Teclas/texto e movimentos individuais não são persistidos; há resumo de atividade a cada 10 s. O teclado não executa comandos automaticamente e o mouse não cria um cursor no OLED.
- Um pendrive MSC: montagem FAT16/FAT32 em /usb, **sem formatação automática** e sem escrita de arquivos pelo aplicativo. ExFAT/NTFS não são habilitados. Remoção encerra a montagem; falha inicial aguarda desconectar/reconectar para tentar montar novamente.
- A raiz fica em cache, com até 32 entradas e nomes de até 80 caracteres. **Reler raiz do pendrive** solicita nova leitura na tarefa USB; arquivos não são copiados para o SD.

Usa as classes USBHost, USBHostHIDKeyboard, USBHostHIDMouse e USBMSCFS do pacote Arduino ESP32 **3.3.12**, com suporte a hub já presente. Se o IDE estiver configurado para iniciar funções TinyUSB Device no boot em modo USB-OTG, o serviço informa conflito em vez de disputar o controlador. Use **USB Mode → Hardware CDC and JTAG**; mantenha CDC/MSC/DFU OTG Device no boot desativados quando aplicável. Não confundir um pendrive conectado como host com apresentar o SD ao PC como USB MSC Device: esta última função não foi adicionada.

Comandos e API

- **usb_status**: estado do host (1 ativo, 2 falha de memória/tarefa, 3 pacote sem suporte, 4 conflito Device, 5 falha de inicialização), dispositivos, capacidade e erros.
- **usb_list**: listagem em cache da raiz do pendrive montado.
- **usb_rescan**: solicita releitura da raiz; conferir usb_list após alguns instantes. Não é reenumeração completa nem repetição da montagem FAT que falhou.
- **log_status**: fila, gravações, descartes e política de rotação.

Funcionam na Serial, UDP e terminal web. Rotas GET sem cache: `/api/usb/status`, `/api/usb/files`, `/api/log/status`. Releitura na página usa o endpoint de comandos já existente, POST protegido pelo header X-ESP32-Command. As operações de pendrive/dispatch HID rodam em tarefa própria, fora do atendimento HTTP e sem gravação SD em callbacks USB.

O que passa a ser registrado no SD

- Boot, versão e motivo do reset; montagem inicial do SD.
- Eventos Wi-Fi/Ethernet, IP Wi-Fi obtido, comandos e metadados de recebimento/resposta UDP.
- Acessos HTTP e códigos/duração das respostas, sem corpo, argumentos de formulário, query string ou conteúdo de imagens. /metrics e /api/dashboard não geram uma linha por consulta para evitar registros repetitivos da própria coleta.
- Conexão/desconexão/erros MQTT com códigos de transporte/broker; nenhum payload de credenciais.
- Inicialização/falhas da câmera, início/fim dos streams, consulta Open-Meteo e falhas de áudio. Quadros não são gravados como binário no log.
- Início/fim/falhas de OTA, alertas/recuperações edge já existentes, conexão/remoção HID/MSC e resultado da montagem/leitura do pendrive.
- A cada 60 s: heap, mínimo de heap, PSRAM, temperatura CPU, tarefas, Wi-Fi/Ethernet/MQTT/RSSI, contadores UDP/MQTT/vídeo/USB e estado/capacidade/erros do SD.
- Avisos/erros do SDK emitidos pelo hook esp_log_set_vprintf, quando habilitados na compilação. Logs INFO/DEBUG do SDK, dumps de relatórios HID, early logs ROM/ISR, todas as chamadas Serial.print e conteúdo bruto de arquivos/imagens não são espelhados indiscriminadamente. Os eventos explícitos acima são registrados independentemente do nível INFO do SDK.

As mensagens explícitas entre tarefas passam por fila de **64 entradas** (até 199 bytes de texto por entrada), com uptime do evento. O ciclo da aplicação drena até quatro por segundo para limitar trabalho de I/O. A gravação inclui carimbo local, quando houver NTP; sem NTP, mantém indicação de ausência de hora e o uptime. Escritas do próprio logger suprimem a captura SDK para evitar recursão de erros SD. Há filtro de palavras relacionadas a senha/autorização/token e remoção de caracteres de controle; respostas com credenciais de OTA não são copiadas da Serial para o arquivo.

Isso não é garantia de registrar cada pacote ou cada evento em alta frequência. Fila cheia, falha/ausência do SD e perda de alimentação podem descartar mensagens; confira log_status e as métricas. Sem SD no boot, a fila é limitada e não há remontagem automática. A escrita é pausada enquanto a flash está em atualização, com tentativa limitada de drenagem após a conclusão; eventos ainda na RAM podem se perder ao reiniciar.

Rotação e leitura

O arquivo atual continua **/log.txt**. Ao atingir 1 MiB, gira para **/log.1.txt** e mantém quatro históricos, até **/log.4.txt**; o mais antigo é removido na próxima rotação. Em operação normal, isso limita os logs a aproximadamente **5 MiB**, sem contar relatórios de crash ou outros arquivos do cartão. A rotação não formata o SD e verifica renomeações; falha interrompe a gravação daquele registro.

A página **Log do dispositivo** continua lendo o arquivo atual. Os arquivos históricos podem ser vistos/listados com os comandos SD existentes. log_status.written conta apenas a fila nova; sdcard_log_writes_total inclui também gravações diretas preexistentes.

Prometheus adiciona **usb_host_up**, **usb_hid_reports_total**, **sdcard_log_queue_dropped_total** e **sdcard_log_rotations_total**. Os contadores reiniciam no boot. Preferências, streaming, MQTT e partições OTA de 16 MB permanecem configurados como antes.

Validação: **75 testes de interface/partições passaram**. Incluem estado USB/FAT/fila, nomes de arquivos como texto, releitura com header protegido, falhas e bloqueio de consultas sobrepostas. Não houve compilação, montagem real de pendrive, teste físico de hub/HID, teste de carga ou de rotação em SD. Após compilar/gravar, conferir usb_status e log_status, conectar os dispositivos no grupo selecionado pelo jumper e verificar /log.txt.

Mapa de hardware: pinos utilizados e esquema elétrico das ligações.

Estabilidade da coleta durante vídeo — 1.19.1

Após o relato de interrupção no Home Assistant e Prometheus, a revisão remove o envio de JPEG do ciclo principal da web. Em 1.19.0, /camera.jpg ainda transmitia a imagem inteira por WebServer; um destinatário lento podia atrasar o atendimento de métricas e a publicação MQTT. Isso foi identificado no código, mas a causa completa do relato ainda depende de logs da placa.

- **/camera.jpg na porta 80 agora retorna HTTP 302** para **http://IP_DA_PLACA:PORTA_VIDEO/snapshot**. O cliente deve seguir o redirecionamento. O JPEG é enviado por uma tarefa do servidor da câmera. Para o Home Assistant, pode informar diretamente essa URL /snapshot na porta do vídeo, evitando o redirecionamento.
- O servidor admite até **dois streams + um snapshot em envio**; fotos simultâneas adicionais recebem 503 em vez de criar tarefas ilimitadas. O JPEG /snapshot permite CORS somente para leitura de imagem pelo navegador da porta 80; as APIs de configuração/comandos não mudam.
- Requests assíncronos concluídos liberam a posse e solicitam fechamento da sessão TCP, para recuperar os sockets.
- O limite de captura/envio foi reduzido de **5 para 2 imagens/s**, preservando resolução e qualidade JPEG 70. O timeout de rede continua em 2 s por operação; isso não é um prazo total para a sessão de vídeo.
- camera_status/API incluem **snapshot_clients**, **rejected_clients**, **heap_free** e **psram_free**, além dos contadores de streaming existentes. Recusas por limite não significam perda de pacote de MQTT/Prometheus.

No Home Assistant (MJPEG IP Camera), mantenha **MJPEG URL = http://IP_DA_PLACA:PORTA_VIDEO/stream** e use **Still Image URL = http://IP_DA_PLACA:PORTA_VIDEO/snapshot**. A porta normal é 81; confira o valor mostrado em /camera ou camera_status. Os endereços do broker MQTT e do alvo Prometheus permanecem os anteriores.

Validação: **69 testes de interface/partições passaram**. Não houve compilação, teste de carga da rede, conexão real ao Home Assistant/broker ou confirmação física da correção. Para validar na placa: com vídeo aberto, conferir /metrics na porta configurada e mqtt_status; depois fechar o vídeo e verificar a liberação de stream_clients. Se a interrupção persistir, registrar camera_status, mqtt_status e se a web continua respondendo. As preferências e a tabela de partições foram mantidas.

Vídeo MJPEG e câmera no OLED — 1.19.0

A página **Câmera** (`/camera`, porta 80) agora inicia um fluxo MJPEG contínuo, com captura JPEG limitada a **5 imagens/s**, qualidade 70 e resolução do driver. Há botões para iniciar vídeo, obter foto JPEG e pausar/retomar a visualização. Ocultar ou fechar a página encerra a conexão de vídeo do navegador. Falha de conexão gera aviso e nova tentativa; a captura na placa continua independente do navegador.

Servidor de vídeo

O endereço normal é **http://IP_DA_PLACA:81/stream**. Para preservar o Prometheus configurado, o firmware tenta portas TCP de 81 a 89 e pula a porta de métricas salva e portas que já estejam ocupadas. A URL exata aparece na página e em `camera_status` / `GET /api/camera/status` (stream_port e stream_path). Se todas estiverem ocupadas, stream_port é 0 e o JPEG continua disponível. Uma alteração no ambiente/porta do

Prometheus pode mudar a porta de vídeo no próximo boot; confira a URL antes de cadastrar no Home Assistant. Não há nova preferência de porta de streaming.

O servidor de vídeo usa ESP HTTP Server com handlers assíncronos e uma tarefa por espectador, no máximo **dois clientes simultâneos**. Um terceiro recebe 503. O stream usa multipart/x-mixed-replace com frames image/jpeg, sem áudio. Cada cliente copia um JPEG recente para a PSRAM, libera o cache e envia pela própria tarefa, sem manter mutex durante a transferência. Timeout de envio de 2 s encerra clientes lentos; FPS real depende de rede, cena e processamento. Web/UDP/MQTT/Prometheus continuam em seus serviços anteriores. A porta ativa do vídeo passa a ser recusada na configuração do Prometheus para evitar colisão.

Durante OTA, a pausa da captura encerra streams existentes; snapshots e novas conexões sem imagem disponível recebem 503. Retomada do navegador usa reconexão automática. Não há gravação de vídeo no SD, envio das imagens por MQTT ou RTSP/H.264 nesta revisão.

Home Assistant

Em **Configurações → Dispositivos e serviços → Adicionar integração → MJPEG IP Camera**, configure:

- **MJPEG URL:** http://IP_DA_PLACA:81/stream — substitua pela porta exibida na página.
- **Still Image URL:** http://IP_DA_PLACA/camera.jpg.
- Usuário e senha vazios, como no JPEG anterior.

Depois adicione a nova entidade ao painel. O Home Assistant e um navegador podem consumir as duas vagas de streaming. Referência: integração MJPEG IP Camera.

Comando camera_oled

Envie **camera_oled** sem argumentos pela Serial, UDP ou terminal web; há também o atalho **Câmera no OLED** no terminal. O Home Assistant ganha um sexto botão MQTT, **Camera no OLED**, que usa o mesmo comando restrito.

O worker reduz o quadro RGB565 para **128×64 preto e branco**, com dithering e recorte central 2:1, preenchendo o SSD1306. Parte das bordas do enquadramento original pode ser cortada. O comando copia um quadro recente e o mostra por **10 segundos**, voltando depois às telas. GPIO27 ou tela:next sai antes. Outros comandos, mensagens de sistema, atualização de firmware ou ações de tela podem substituir a imagem. Não é vídeo no OLED e não muda as preferências do protetor/rotação. OLED ausente/ocupado ou câmera sem imagem recente retornam diagnóstico sem desligar a aplicação.

Diagnóstico e validação

`camera_status` agora inclui stream_port, stream_path, stream_clients, stream_frames e stream_errors. Prometheus adiciona `camera_stream_clients` (gauge), `camera_stream_frames_total` e `camera_stream_errors_total` (counters). Frames contam envios concluídos por cliente; desconexões durante envio e timeouts podem incrementar erros e não significam necessariamente defeito no sensor. Contadores reiniciam no boot. Os frames JPEG da câmera continuam contados separadamente.

Mantidos partitions.csv para flash 16 MB, dois aplicativos OTA de 6,25 MiB e as preferências anteriores. **69 testes passaram** (interface e tabela de partições), incluindo stream com porta alternativa, pausa/visibilidade, snapshots, reserva da porta no Prometheus e atalho camera_oled. Não houve compilação nem teste físico do novo streaming/OLED. O usuário confirmou imagem JPEG boa na versão anterior.

Espaço do firmware e partições — 1.18.1

O erro de compilação com máximo de **1966080 bytes** corresponde ao esquema Minimal SPIFFS (1.9MB APP). O firmware informado tem **2316966 bytes**, portanto esse esquema é pequeno demais; a RAM não é a causa deste erro.

Foi adicionado `partitions.csv` na pasta do sketch para os **16 MB de flash da ESP32-P4-Module-DEV-KIT**. Cada aplicativo OTA tem **6553600 bytes (6,25 MiB)**. Mantém NVS, otadata, dois aplicativos OTA, SPIFFS e coredump. A tabela coincide com default_16MB.csv do pacote Arduino ESP32 3.3.12. A PSRAM de 32 MB é memória de trabalho, não espaço para o firmware.

No Arduino IDE:

1. **Tools / Ferramentas → Flash Size → 16MB (128Mb)**.
2. **Partition Scheme → Custom**. O arquivo partitions.csv precisa permanecer junto do wifi_Eth_SDCard.ino. A opção Custom do pacote 3.3.12 elimina o antigo limite de 1.9 MB no verificador do IDE; o limite real continua sendo 6553600 bytes por aplicativo.
3. Mantenha **PSRAM habilitada** para a câmera.
4. Faça a primeira gravação dessa tabela pela **porta USB/serial**, não por OTA. A atualização de aplicativo pela rede não instala a nova tabela de partições. Depois disso, os próximos aplicativos podem voltar a ser enviados por OTA.

Não selecione Huge APP/No OTA, pois o projeto usa duas áreas de aplicativo. Não é necessário apagar toda a flash: deixe **Erase All Flash Before Sketch Upload → Disabled**, se disponível. NVS mantém endereço 0x9000 e tamanho 0x5000, permitindo preservar as preferências ao gravar sem apagar tudo. A localização dos aplicativos, SPIFFS e coredump muda em relação ao esquema antigo; dados antigos dessas regiões não são preservados por esta migração. Arquivos no cartão SD não dependem dessa tabela.

Validação desta revisão: **65 testes passaram** (60 de interface e 5 da tabela de partições), conferindo alinhamento, ausência de sobreposição, limite físico, dois slots OTA iguais e espaço para o tamanho de firmware informado. Não houve recompilação ou gravação da placa.

Clima externo — 1.15.0

Temperatura e umidade de Porto Alegre vêm do Open-Meteo, nas coordenadas -30.03283, -51.23019, iguais às do M5NanoC6. São condições estimadas pelo serviço meteorológico; a temperatura interna da CPU permanece separada.

A primeira consulta aguarda internet por Wi-Fi ou Ethernet e relógio sincronizado por NTP. Atualiza a cada 10 minutos; em caso de falha, tenta novamente após 2 minutos. O cache expira em 30 minutos, considerando também o horário informado pela API; dados ausentes ou vencidos viram `NaN`. A consulta HTTPS roda em tarefa própria no core 0, com resposta limitada a 2048 bytes. `/metrics` lê somente o cache. Novas consultas são suspensas durante OTA; uma consulta já iniciada pode terminar.

Use `weather_status` pela Serial, UDP ou terminal web para ver rede, NTP, validade, temperatura, umidade, idade e último resultado HTTP. Instale **ArduinoJson 7.x** (revisado com os cabeçalhos 7.4.3); HTTPClient e NetworkClientSecure acompanham o pacote Arduino ESP32.

Revisão documental: **31/10/2026**.

Estado conferido com o código **1.20.0**: GPIO27 para navegação, protetor de robôs, som de troca de página e o comando `reset_counters`. A compilação e a gravação permanecem pendentes; esta revisão também altera o firmware.

Desde 1.13.2, o comando `temp` compartilha o sensor com a métrica `esp_cpu_temperature_celsius` do Prometheus/Grafana. Em falha, responde `Temp: indisponivel`; a métrica fica `NaN`, sem retornar zero falso. CHANGELOG.md e catálogo geral.

A resposta de `help` começa com `COMANDOS` e apresenta um comando por linha. Argumentos aparecem após `:`; descrições e exemplos ficam neste guia, fora da lista parseável.

Envie por UDP à porta **4210**, pela serial a **115200 baud** ou pelo terminal web em `/info`. O firmware responde pelo canal de origem. Na serial, termine cada comando com LF, CR ou CRLF e aguarde o fim da inicialização. Limite: **254 bytes por comando**. Nomes não distinguem maiúsculas/minúsculas; argumentos preservam a caixa. Arquivos SD podem gerar várias respostas UDP.

Prometheus pela interface web

O padrão de conexão segue o M5NanoC6. Configure endereço do servidor, porta de métricas da placa e intervalo (5–3600 s) em /prometheus, na porta 80. As Preferences preservam esses valores. A porta muda após cerca de um segundo, sem reiniciar; a web e a API permanecem na 80.

Na porta 80, /metrics compartilha a web. Nas outras portas, o servidor de coleta atende apenas /metrics. A porta 3232 é reservada ao OTA. Não há envio ativo. Não há mais chave de ativação ou campo para porta de acesso ao servidor Prometheus.

Endpoint na porta 80	Ação
GET /prometheus	Formulário de conexão e YAML gerado.
GET /api/prometheus/config	host, port, active_port, web_port, interval, mode, scrape_config, persistent_counters_ok.
POST /api/prometheus/config	Salva formulário host, port e interval; exige X-ESP32-Command: 1.
GET /prometheus.yml	Download do YAML para aplicar no servidor.

POST /api/prometheus é um alias que aceita os mesmos três campos do novo formato. GET /metrics na porta 80 retorna 404 quando outra porta de métricas está ativa. As métricas, inclusive as do SD, são descritas em PROMETHEUS.md. O padrão atual da porta de métricas é **80**; valores legados salvos são preservados na migração. O namespace deste projeto é `prometheus`, não `telemetry`. O endereço do servidor é uma referência, não uma restrição de origem. `reset_counters` zera somente os contadores de boot, reinicialização e watchdog nesse namespace, preservando a configuração de coleta.

Diagnóstico de áudio — 1.14.1

Execute audio_test, aguarde cerca de um segundo e consulte audio_status. O status confirma dados enviados ao I2S, não a saída acústica. Escritas parciais/timeout são registradas e a próxima solicitação pode tentar novamente. A preparação inclui cerca de 100 ms de silêncio para o amplificador. Veja README.md para detalhes.

OLED, botão e áudio

Botão GPIO27/GND (INPUT_PULLUP), ativo em LOW: uma pressão acorda e avança uma página; segurar não apaga Wi-Fi nem repete. A rotação continua a cada oito segundos enquanto o protetor está inativo. Após 60 s sem interação, o robô ocupa todo o OLED, sem desligar a tela. Logs e consultas web/Prometheus não reiniciam a inatividade. Trocas manuais e automáticas entre páginas emitem um bip assíncrono pelo ES8311.

Comando	Função
protetor:on	Mostra as carinhas agora.
protetor:off	Acorda e desativa a entrada automática.
protetor:auto	Acorda e reativa a entrada automática.
protetor:status	Consulta sem acordar ou reiniciar a inatividade.
protetor:tempo:60	Tempo de inatividade, de 5 a 3600 s; fica em RAM.
audio_status	Estado do ES8311/I2S, pinos, erro, contadores, bytes enviados e leitura dos registradores.
audio_test	Solicita teste de 500 ms / 880 Hz sem trocar a página; não recebe argumentos.
botao_status	Pino configurado e leitura LOW/HIGH.

tela:next e tela:auto também saem das carinhas. O terminal web oferece o atalho Carinhas de robô. A configuração detalhada de som está em README.md. Não há handler sd_test nesta versão.

Exemplos para conferir após gravar o firmware:

```
botao_status
audio_test
audio_status
protetor:on
tela:next
protetor:tempo:60
protetor:auto
```

`botao_status` deve indicar GPIO27 e LOW ao pressionar. `audio_status` mostra se o codec/I2S ficou pronto e o último erro. `protetor:on` permite observar as carinhas sem aguardar o tempo de inatividade; `tela:next` acorda e avança. Os ajustes do protetor ficam em RAM: ao reiniciar, volta a entrada automática após 60 s. O estado das métricas continua independente desses ajustes.

Sistema

Comando	Função	Exemplo
`help`	Lista comandos e aliases.	`help`
`name`	Identifica o dispositivo; responde `ESP32P4`.	`name`

Comando	Função	Exemplo
`info`	Informações gerais da placa.	`info`
`status`	Resumo de Wi-Fi, SD, relógio e heap.	`status`
`version`	Versão do firmware.	`version`
`build`	Data e hora da compilação.	`build`
`crash_info`	Lê o crash, arquiva e verifica no SD, e só então apaga o registro interno.	`crash_info`
`reason`	Motivo do último reset.	`reason`
`alive`	Responde ALIVE.	`alive`
`cores`	Core de execução, tarefa serial e ocupação da fila.	`cores`
`ota_info`	Serviço OTA, porta e tamanho da partição.	`ota_info`
`ota_password`	Mostra usuário `admin` e senha OTA fixa `123456` somente na serial local.	`ota_password`
`uptime`	Tempo ligado em milissegundos.	`uptime`
`lastcmd`	Último comando; credenciais ficam ocultas.	`lastcmd`
`cmdcount`	Total de comandos, exceto lastcmd/cmdcount.	`cmdcount`
`reboot`	Reinicia após responder.	`reboot`
`reset_counters`	Zera somente os contadores NVS de boot/reboot/watchdog.	`reset_counters`

Diagnóstico e hardware

Comando	Função	Exemplo
`health`	Diagnóstico de rede, OLED, SD montado, relógio e memória. Somente leitura.	`health`
`selftest`	Mesmo diagnóstico de health; não realiza teste de escrita ou teste elétrico.	`selftest`
`temp`	Temperatura interna do processador.	`temp`
`cpu`	Modelo, revisão e frequência.	`cpu`
`chip_info`	Modelo, revisão, núcleos, frequência e SDK.	`chip_info`
`ram`	Heap livre, mínimo e maior bloco disponível.	`ram`
`heap`	Heap livre em bytes.	`heap`
`heap_min`	Menor heap livre desde o boot.	`heap_min`
`psram`	PSRAM total e livre em bytes; zero quando indisponível.	`psram`
`flash`	Flash total, tamanho do sketch e espaço de sketch livre.	`flash`

Rede

Comando	Função	Exemplo
`net_info`	IP, gateway, máscara e DNS de Wi-Fi e Ethernet, com estado das interfaces.	`net_info`
`net_monitor`	RTT atual/mínimo/médio/máximo, idade da medição, quedas, tempo offline e eventos.	`net_monitor`
`eth_status`	Estado do cabo, conexão Ethernet, IP, gateway e MAC.	`eth_status`
`wifi_status`	Estado Wi-Fi, código numérico de status/modo, SSID e IP STA/AP.	`wifi_status`
`channel`	Canal Wi-Fi.	`channel`
`mac`	MAC Wi-Fi.	`mac`
`rssi`	RSSI Wi-Fi em dBm.	`rssi`
`ip`	Endereços Wi-Fi e Ethernet.	`ip`
`ssid`	SSID conectado.	`ssid`
`wifi_list`	Somente SSIDs salvos, um por linha; lista vazia responde uma linha vazia.	`wifi_list`
`wifi_add:SSID|SENHA`	Salva/atualiza uma rede; senha vazia cadastra rede aberta. Não reinicia.	`wifi_add:MinhaRede|MinhaSenha123`
`reset_wifi`	Apaga todas as redes salvas e reinicia.	`reset_wifi`

OLED

Comando	Função	Exemplo
`tela:0`	Relógio e data.	`tela:0`
`tela:1`	Wi-Fi: rede, IP, gateway, RSSI e canal.	`tela:1`
`tela:2`	Ethernet: IP, gateway, quedas e tempo offline.	`tela:2`
`tela:3`	Sistema e SD.	`tela:3`
`tela:4`	Ping aos gateways e gráfico.	`tela:4`
`tela:5`	Últimos quatro eventos de conexão.	`tela:5`
`tela:next`	Avança e mantém rotação automática.	`tela:next`
`tela:auto`	Retoma rotação a cada oito segundos.	`tela:auto`

Horário

Comando	Função	Exemplo
`time`	Hora local.	`time`
`date`	Data local.	`date`
`ntp_status`	Validade do relógio, data/hora e servidores configurados; não comprova alcance atual do NTP.	`ntp_status`
`set_fuso:N`	Salva offset UTC inteiro de -12 a +14.	`set_fuso:-3`
`dst_on`	Salva horário de verão de uma hora.	`dst_on`
`dst_off`	Desativa e salva horário de verão.	`dst_off`
`dst:0 ou dst:1`	Define horário de verão explicitamente.	`dst:0`

LED

Comando	Função	Exemplo
`led_on`	Acende e cancela sequência de pisca.	`led_on`
`led_off`	Apaga e cancela sequência de pisca.	`led_off`
`led_blink[:ms]`	Pisca durante a duração indicada, alternando a cada 200 ms, sem bloquear o loop. Padrão 3000 ms, máximo 10000 ms.	`led_blink:5000`

Cartão SD

Comando	Função	Exemplo
`sd_status`	Capacidade total, usada e livre.	`sd_status`
`sd_list`	Lista arquivos na raiz; não recebe argumento de pasta nesta versão.	`sd_list`
`sd_read:arquivo`	Envia o arquivo inteiro em múltiplos pacotes UDP.	`sd_read:/dados.txt`
`sd_write:arquivo:texto`	Grava o texto substituindo o conteúdo.	`sd_write:/dados.txt:Primeira linha`
`sd_append:arquivo:texto`	Acrescenta texto e quebra de linha CRLF; cria o arquivo se necessário.	`sd_append:/dados.txt:Outra linha`
`sd_del:arquivo`	Remove arquivo.	`sd_del:/dados.txt`
`sd_exists:arquivo`	Verifica existência.	`sd_exists:/dados.txt`
`sd_size:arquivo`	Tamanho em bytes.	`sd_size:/dados.txt`

Comando	Função	Exemplo
`sd_tail:arquivo[:linhas]`	Envia as últimas linhas; padrão 10.	`sd_tail:/dados.txt:5`
`sd_log[:linhas]`	Últimas linhas de /log.txt; padrão 10.	`sd_log:20`
`sd_clear_log`	Apaga /log.txt.	`sd_clear_log`
`sd_mkdir:pasta`	Cria pasta; a pasta pai precisa existir.	`sd_mkdir:/dados`
`sd_rmdir:pasta`	Remove pasta vazia; raiz não é permitida.	`sd_rmdir:/dados`

Aliases compatíveis

Alias	Comando equivalente
`init`	`reason`
`flash_info`	`flash`
`restart`	`reboot`
`list`	`sd_list`
`read`	`sd_read`
`write`	`sd_write`
`delete`	`sd_del`
`exists`	`sd_exists`
`size`	`sd_size`
`tail`	`sd_tail`
`sd_info`	`sd_status`
`set_tz`	`set_fuso`
`set_dst`	`dst`

Cadastro circular de redes

```
wifi_add:MinhaRede|MinhaSenha123
wifi_add:RedeAberta|
wifi_list
reboot
```

Novos SSIDs ocupam o próximo perfil circular, começando no perfil 1; após o quinto, a próxima inserção substitui o perfil 1. Atualizar um SSID existente troca a senha sem avançar o cursor. `wifi_add` verifica a gravação e tenta restaurar o perfil anterior se falhar. `reboot` aplica a nova lista na busca de redes.

O cursor UDP é independente da última rede utilizada; o formulário HTTP edita os cinco perfis diretamente e mantém o cursor. O SSID não pode conter `|`; a senha pode conter `|`, pois só o primeiro separador é interpretado. SSID: de 1 a 32 bytes. Senha: de 8 a 63 bytes, 64 caracteres hexadecimais ou vazia para rede aberta. Espaços da senha são preservados; CR/LF finais do comando são removidos. Credenciais não aparecem no terminal, no log SD nem em `lastcmd`.

Operações que alteram dados

`reboot`, `reset_wifi`, `wifi_add`, ajustes de fuso/DST e comandos de escrita, exclusão e pastas SD alteram o dispositivo. O firmware executa diretamente; a confirmação, quando houver, pertence ao aplicativo que envia os comandos. `sd_write` substitui o arquivo, `sd_append` acrescenta uma linha e `sd_clear_log` apaga o log. Operações NVS com vários campos não são atômicas contra falta de energia.

Botão e portal

GPIO27/GND: pressione para acordar e avançar a tela. Segurar não limpa redes. Use reset_wifi para essa operação. `/info` mostra informações; `/wifi` edita os cinco perfis.

Interpretação dos diagnósticos

Ping mede o gateway de cada interface, não o acesso à Internet. Um timeout pode ser bloqueio de ICMP. Estatísticas do monitor ficam em RAM e reiniciam no boot. `ntp_status` consulta a validade do relógio local, sem disparar nova consulta NTP. `health` e `selftest` consultam estados já disponíveis; não testam escrita no SD nem garantem a integridade física do hardware. Algumas operações SD continuam bloqueantes e podem atrasar telas e monitor.

Serial USB e terminal web (v1.8.0)

Todos os comandos deste documento também funcionam na serial, a **115200 baud**, e no terminal da página `/info`. Na serial, envie uma linha por comando com final LF, CR ou CRLF. Aguarde o fim da inicialização da placa. Limite: **254 bytes**. Para a porta USB nativa, habilite **USB CDC On Boot** conforme a configuração da placa.

As respostas são enviadas ao canal de origem. No terminal web, respostas acima de 8 KB são truncadas com um aviso; use `sd_tail` para consultar arquivos grandes. O terminal pede confirmação para comandos que alteram arquivos/configurações. Se a conexão cair, verifique o estado antes de repetir um comando de escrita.

crash_info — 1.12.0

O comando `crash_info` segue o central: informa o motivo do último reset, lê o core dump válido, arquiva o relatório no SD e só apaga o registro interno após reabrir o arquivo e conferir tamanho e conteúdo byte a byte. Sem SD, com gravação incompleta ou falha na conferência, o registro interno permanece para nova tentativa.

O relatório mostra panic, tarefa, PC e identificador SHA256 do ELF. No ESP32-P4 (RISC-V), inclui RA, SP, MCAUSE, MTVAL e a pilha bruta fornecida pelo SDK; a decodificação do backtrace exige o ELF do firmware que sofreu o crash. Preserve esse arquivo junto de cada versão. Não são exibidos endereços da pilha como se fossem um backtrace já decodificado.

Os arquivos são `/crash_YYYYMMDD_HHMMSS.txt` ou `/crash_SEM_NTP_<uptime>.txt`, com sufixo numérico quando necessário. Use `sd_list` e `sd_read:/crash_....txt` para consultá-los. A página `/logs` continua lendo apenas `/log.txt`.

Disponível pela serial, UDP e terminal web. No UDP, o relatório é dividido em pacotes de até 1200 bytes para caber no receptor do central; a entrega continua sujeita às perdas e à ordem do UDP. No navegador, a operação pede confirmação, pois arquiva dados e apaga o registro interno depois da verificação.

Sem crash válido, informa essa condição e o último reset, sem criar arquivo ou apagar dados. Se o SDK não habilitar core dump em flash/ELF, informa que o recurso está desabilitado. O SDK ESP32-P4 3.3.12 e a tabela padrão usados no projeto já incluem esse suporte e a partição coredump. O comando não provoca um crash.

Métricas UDP — 1.15.1

Métrica	Significado
`esp_udp_listener_up`	Gauge: 1 se a última abertura da porta UDP4210 retornou sucesso; 0 antes da abertura ou após falha. Indica abertura local, não conectividade de ponta a ponta.
`esp_udp_packets_received_total`	Counter: datagramas identificados por parsePacket, incluindo comandos inválidos e maiores que o limite. Datagramas vazios não são detectáveis por esse retorno.
`esp_udp_replies_sent_total`	Counter: respostas UDP com todos os bytes escritos e endPacket bem-sucedido. Não confirma recebimento pelo destinatário. Cada chamada de resposta conta separadamente.
`esp_udp_send_errors_total`	Counter: tentativas de resposta sem destinatário válido, falha ao iniciar/concluir envio ou escrita incompleta. Uma falha conta uma vez por tentativa.

Os três contadores reiniciam no boot, sem persistência NVS, e não são alterados por reset_counters. Respostas HTTP e Serial não os incrementam. Perder o link não fecha automaticamente o socket nem muda o resultado da última abertura.

No Grafana: `rate(esp_udp_packets_received_total[5m])` para pacotes/s e `increase(esp_udp_send_errors_total[1h])` para erros na última hora.

MQTT e Home Assistant — 1.16.0

Abra **MQTT / Home Assistant** no menu web (`/mqtt`, porta 80), informe o endereço do broker sem prefixo mqtt://, porta (padrão 1883), usuário e senha e marque **Ativar MQTT**. O recurso inicia desativado. Configure a integração MQTT do Home Assistant com o mesmo broker; o firmware é cliente, não instala nem executa um broker.

A configuração fica no valor `config` do namespace Preferences `mqtt`, com verificação de escrita. A aplicação ocorre após aproximadamente um segundo. Senha vazia mantém a anterior; **Apagar senha salva** remove a senha. A senha não aparece na página, no JSON de consulta ou em mqtt_status. Transporte TCP sem TLS e armazenamento nas preferências da placa: utilize na rede local.

Usa ESP-MQTT já incluído no pacote Arduino ESP32 3.3.12, sem PubSubClient. A conexão/reconexão roda na tarefa interna do cliente; publicações comuns usam enqueue, com outbox limitado a 16 KiB. Ao alterar/desativar a configuração, o encerramento da conexão anterior pode esperar uma operação de rede por até o timeout de 5 s. Não há compilação ou gravação automática.

Entidades e tópicos

Descoberta retida sob `homeassistant/<componente>/<id>/<entidade>/config`, com identificação derivada do MAC de fábrica. O tópico base é `esp32/esp32p4_<id>`; confira o valor exato na página ou em `mqtt_status`. Cada placa possui um ID próprio. Descoberta é reenviada na reconexão e quando Home Assistant publica online em `homeassistant/status`. Configure o prefixo de descoberta padrão homeassistant.

- Sensores: temperatura CPU, temperatura e umidade externas de Porto Alegre (Open-Meteo), uptime, heap livre, RSSI Wi-Fi, SD livre e usado. Valores indisponíveis são null no JSON, apresentados como desconhecidos.
- Estado binário do SD, interruptor do LED e cinco botões: próxima tela, telas automáticas, ligar/desligar carinhas e testar som.
- Evento **Botao GPIO27** com tipo `press`: uma publicação por pressão após debounce, preservando a navegação no OLED. Eventos não são retidos nem armazenados enquanto offline.
- `<base>/state`: JSON retido a cada 30 s e após comandos. Sensores numéricos expiram após 120 s sem atualização.
- `<base>/availability`: online/offline retido, com Last Will QoS 1 e keepalive 30 s. Queda inesperada é detectada pelo broker, portanto não é instantânea.
- `<base>/command`: aceita somente `led_on`, `led_off`, `tela:next`, `tela:auto`, `protetor:on`, `protetor:off` e `audio_test`. Envie sem retain. Comandos retidos, fragmentados, maiores que 31 bytes ou fora da lista são ignorados. A fila comporta oito comandos. Não expõe reinício, exclusão de arquivos ou credenciais.

Durante OTA/reinício agendado, publicações periódicas e execução de comandos ficam suspensas; comandos recebidos são descartados. A conexão interna MQTT pode permanecer ativa. Não há histórico offline de amostras: após reconectar publica o estado atual. Desativar MQTT mantém as entidades descobertas no Home Assistant, indisponíveis após o aviso offline ou expiração dos sensores; para removê-las, exclua as configurações retidas correspondentes no broker.

API e diagnóstico

`GET /api/mqtt/config` retorna enabled, host, port, user, password_set, connected, base_topic e contadores, nunca a senha. `POST /api/mqtt/config` recebe formulário enabled (0/1), host, port, user, password opcional e clear_password (1 para apagar), com header `X-ESP32-Command: 1`. Respostas: 400 inválido, 409 durante OTA/reinício/aplicação anterior, 503 falha NVS. As rotas retornam no-store e não habilitam CORS.

O comando `mqtt_status` funciona na Serial, UDP e terminal web. No Prometheus, `mqtt_connected` indica a conexão; `mqtt_messages_sent` conta publicações QoS 1 confirmadas pelo broker (inclui

descoberta/estado/eventos), `mqtt_messages_received` conta mensagens completas/iniciadas recebidas incluindo anúncios do Home Assistant e comandos ignorados; `mqtt_publish_errors` conta falhas locais, conexão, assinatura, fila e descarte de publicações pelo cliente. Os três são counters e reiniciam no boot, sem persistência e sem alteração por reset_counters. `esp_metric_available{group="mqtt"}=1` indica suporte implementado, mesmo desativado.

Validação desta revisão: **44 testes da interface passaram**; não houve compilação, conexão real ao broker ou teste físico. Após compilar/gravar, salve o broker, confirme connected em mqtt_status e confira o dispositivo ESP32-P4 na integração MQTT do Home Assistant. Teste o LED, os controles de tela e o evento físico GPIO27. Referência: MQTT no Home Assistant.

Processamento local (edge computing) — 1.17.0

Abra **Processamento local** no menu web (`/edge`, porta 80), ajuste os limites e marque **Ativar regras locais**. O recurso inicia desativado e o som também. As decisões usam dados locais da placa e não dependem de internet, NTP, MQTT ou Prometheus. É um motor de regras determinístico, sem modelo de IA.

Regra	Condição inicial	Recuperação
`cpu_hot`	CPU maior ou igual a 70 °C	CPU menor ou igual ao limite menos 3 °C
`heap_low`	Heap livre abaixo de 65536 bytes	Heap maior ou igual ao limite mais 8192 bytes
`network_down`	Sem Wi-Fi STA e sem Ethernet com IP	Uma das duas redes volta; não testa acesso à internet
`udp_errors`	Pelo menos 5 falhas de envio UDP nos últimos 60 s	Quantidade na janela cai abaixo do limite

Amostragem uma vez por segundo no ciclo da aplicação, sem espera ativa. Rede e UDP aguardam 60 s após boot/ativação/retorno de OTA para formar a janela inicial. A janela UDP usa 60 posições por segundo e o contador esp_udp_send_errors_total; não detecta por si só perda de pacotes ou ausência de respostas remotas. Uma pausa de amostragem de 60 s descarta a janela anterior e recomeça o aquecimento.

Confirmação padrão: 10 s contínuos, tanto para ativar quanto para recuperar. Oscilações ou dado inválido cancelam a confirmação pendente. CPU inválida não cria um zero e não encerra um alerta existente; a regra fica indisponível até nova leitura válida. **Intervalo mínimo: 300 s** entre notificações de alerta da mesma regra; condições persistentes repetem a notificação após esse prazo. Recuperações são notificadas após confirmação, sem esperar o intervalo. O estado ativo pode mudar mesmo quando uma nova notificação está limitada pelo intervalo.

Ações e histórico

As notificações e recuperações aparecem na Serial e em `/log.txt` quando o SD aceita a escrita. A página mostra os últimos **16 eventos em RAM**, com sequência e uptime em segundos. Um reinício limpa esse histórico e os contadores, mas não apaga os logs já gravados. Cada regra pode produzir alerta repetido no intervalo configurado. Se **Emitir som nos alertas** estiver marcado, solicita o tom de teste ao sistema de áudio existente; indisponibilidade ou fila ocupada pode impedir o som sem impedir o registro. Não reinicia a placa, não apaga preferências e não altera o LED ou a navegação automaticamente.

Durante OTA ou reinício agendado, a avaliação e suas ações ficam pausadas. Ao retomar, confirmações e janela UDP são reiniciadas; os estados ativos anteriores permanecem até reavaliação válida. Salvar configuração reinicia a janela e limpa estados ativos/pendentes sem criar eventos de recuperação; mantém contadores e histórico do boot. A configuração é salva em um valor `config` no namespace Preferences `edge`, com conferência da escrita.

MQTT / Home Assistant

Se MQTT estiver conectado, eventos são enviados sem retain em `<base>/edge/event`: JSON com rule, active, sequence e uptime. Não há fila histórica de eventos gerados offline; o estado atual segue no JSON `<base>/state` (edge_alert ON/OFF, edge_enabled e edge_active_count), publicado no máximo a cada 30 s e solicitado imediatamente após uma notificação. O Home Assistant descobre o sensor binário **Alerta local** no mesmo dispositivo MQTT. Regras continuam funcionando com o broker desligado.

API, diagnóstico e métricas

- `GET /api/edge/config`: enabled, sound, cpu_limit, heap_limit, udp_limit, confirm_seconds, cooldown_seconds.
- `POST /api/edge/config`: os mesmos campos como formulário; enabled/sound são 0 ou 1 e é obrigatório o header `X-ESP32-Command: 1`. Limites: CPU 20–100 °C; heap 8192–1048576 bytes; UDP 1–10000 falhas/60 s; confirmação 1–600 s; intervalo 30–86400 s. Retorna 400 inválido, 409 OTA/reinício ou 503 NVS. Rotas sem cache e sem CORS.
- `GET /api/edge/status` e comando `edge_status` (Serial, UDP e terminal web): configuração, amostras locais, pausa, estado/validade/confirmação de cada regra e histórico.
- Prometheus: `esp_edge_enabled`, `esp_edge_active_alerts`, `esp_edge_events_total`, `esp_edge_rule_active{rule="..."}`, `esp_edge_rule_available{rule="..."}`, `esp_edge_rule_alerts_total{rule="..."}`. Eventos contam alertas/repetições e recuperações; o contador por regra conta somente notificações de alerta. Contadores reiniciam no boot e não são alterados por reset_counters. Disponibilidade 0 significa regra desativada, em aquecimento ou sem leitura válida. Estado ativo anterior pode permanecer com disponibilidade 0.

Validação desta revisão: **53 testes da interface passaram**. Incluem configuração, limites, falhas de persistência simuladas e atualização do estado sem sobrescrever edições. Não houve compilação, teste do motor C++ em execução ou teste físico. Após compilar/gravar, ative as regras pela web, confira edge_status e valide alertas/recuperações controlados na placa.

Câmera OV5647 MIPI-CSI — 1.18.0

A câmera confirmada para esta configuração é **OV5647 no conector MIPI-CSI com cabo flat**. O firmware tenta inicializá-la automaticamente no boot. Abra **Câmera** no menu web (`/camera`, porta 80). A página consulta uma imagem JPEG por segundo; o botão Pausar suspende somente a visualização, não a captura na placa. A imagem anterior recebe aviso de desatualização se a consulta falhar.

Captura e integração

- Usa ESP_Video e esp_video já presentes no Arduino ESP32 3.3.12, com driver OV5647/ISP e encoder JPEG do ESP32-P4. Não usa o esp_camera dos módulos ESP32-CAM DVP.
- Compartilha o handle I^2C da Wire no barramento 0 (SDA7/SCL8) com OLED e ES8311. Não cria outro barramento nem altera os pinos do botão/áudio. O sensor usa SCCB a 100 kHz.

- Captura RGB565 com dois buffers; converte em tarefa própria no core 0 para JPEG de qualidade 70, limitado a duas codificações por segundo. A resolução vem do driver instalado (OV5647 padrão 800×800 neste pacote); aceita até 1280×1280, com dimensões múltiplas de 16.
- Requer PSRAM habilitada. Reserva dois buffers de saída JPEG de aproximadamente largura×altura×3+1024 bytes cada, além dos buffers do driver de vídeo. Falha de inicialização libera os recursos da câmera; o restante da aplicação continua. Não há repetição automática da inicialização: confira camera_status e reinicie após corrigir a causa.
- O servidor copia o JPEG mais recente com proteção de acesso; não captura/espera pelo sensor dentro da requisição web. Falta de memória, cache ocupado ou imagem com mais de 3 s retorna HTTP 503. A transferência HTTP do JPEG ainda ocupa o handler durante o envio.
- OTA pausa novas capturas/codificações do worker e bloqueia snapshots; uma operação já iniciada pode terminar. O pipeline CSI/ISP pode permanecer ativo. Câmera sem responder pode deixar somente o worker aguardando frame, sem esperar no ciclo da web.

Rotas, diagnóstico e métricas

`GET /camera.jpg`: JPEG atual sem cache, para abrir no navegador ou usar como URL de imagem em outra integração. Não é fluxo MJPEG, RTSP ou gravação de vídeo. `GET /api/camera/status` e comando `camera_status` (Serial, UDP e terminal web) mostram inicialização, disponibilidade, pausa, resolução, quantidade de JPEGs, falhas, idade da última imagem e diagnóstico de captura. `ready` indica inicialização concluída; `available` exige também JPEG recente e ausência de pausa.

Prometheus inclui `camera_initialized`, `camera_up`, `camera_frames_total`, `camera_errors_total` e `camera_frame_age_seconds`. Frames contam JPEGs disponibilizados no cache, não todos os frames do sensor. Falhas contam inicialização, frame inválido/incompleto e erro/timeout de codificação. Idade é NaN antes do primeiro JPEG. Contadores reiniciam no boot. Imagens não são publicadas por MQTT nem gravadas automaticamente no SD; a integração MQTT anterior continua ativa.

Validação desta revisão: **60 testes da interface passaram**, incluindo atualização, pausa, erro JPEG, descarte de URLs antigas e bloqueio de consultas sobrepostas. Não houve compilação nem teste físico da câmera. Após compilar/gravar, abra /camera e confira camera_status; se não houver imagem, verifique a detecção do sensor, cabo CSI e alimentação. Referência da placa: Waveshare ESP32-P4-Module-DEV-KIT.

Diagnóstico MQTT em 1.19.1: mqtt_status e GET /api/mqtt/config incluem disconnects, last_error_type, broker_return_code, socket_errno, transport_error, heap_free e network_up. O tipo 1 identifica transporte TCP; tipo 2 indica recusa do broker (código 4: credenciais inválidas, 5: acesso não autorizado). Os erros de conexão são limpos após uma reconexão bem-sucedida; o contador errors continua acumulado e inclui outras falhas locais. Um contador errors alto, isoladamente, não comprova senha incorreta ou falta de sockets. O usuário confirmou enabled=true, connected=false, sent=204 e errors=575 na versão anterior; isso confirma comunicação anterior e desconexão atual, sem identificar a causa final.